Лекция 16. Архитектурный разбор, отказы и ретроспектива MLOps-системы

Версия: 2026–2027 учебный год Продолжительность: 2 академических часа Связанная работа: ЛР16 «Интеграция, release manifest и защита проекта» Формат: итоговая лекция, разбор кейса и подготовка к экзаменационному контролю

1. Цель лекции

Собрать знания курса в целостную модель эксплуатации ML-системы: научить студентов читать архитектуру через риски и доказательства, локализовать отказ, выбирать rollback или roll forward, проводить содержательный postmortem и обосновывать дальнейшее развитие системы.

2. Результаты обучения

После лекции студент сможет:

- описать полный контур от данных до решения пользователя;
- выделить архитектурные инварианты и владельцев компонентов;
- классифицировать отказ по слою, влиянию и обнаружению;
- отличить симптом, непосредственную причину и системные факторы;
- предложить безопасный порядок диагностики и восстановления;
- связать incident evidence с release manifest и наблюдаемостью;
- оформить blameless postmortem с проверяемыми action items;
- оценить технический долг и приоритет следующего улучшения;
- защитить инженерное решение, альтернативу и остаточный риск.

3. Финальная лекция — не каталог инструментов

За семестр были использованы GitHub, DVC, MLflow, pytest, Ruff, FastAPI, Docker, Prefect, Prometheus и Grafana. Знание команд этих инструментов полезно, но итоговая компетенция выше: студент должен понимать, какое утверждение доказывает каждый артефакт и что произойдёт при его отказе. Инструмент можно заменить; контракт, identity, gate и обратная связь должны сохраниться.

4. Сквозной объект управления

ML-система принимает не только HTTP-запрос. Она принимает данные, код, зависимости, конфигурацию, решения людей и время. Выходом является не только prediction, а действие пользователя или бизнес- процесса. Поэтому граница анализа проходит от provenance данных до наблюдаемого последствия решения.

5. Архитектурные инварианты курса

Инвариант — утверждение, которое должно оставаться истинным при изменении реализации:

- данные и признаки доступны к моменту прогноза;
- любой результат связан с версией кода, данных и параметров;
- модель выпускается только после объявленных gates;
- API проверяет контракт до вызова модели;
- image и model version идентифицируются неизменяемо;

- сервис сообщает readiness и ключевые SLI;
- решение о выпуске и исключения имеют владельца;
- восстановление возвращает совместимое проверенное состояние;
- личный вклад и review прослеживаются по evidence.

Если команда не может быстро показать доказательство инварианта, инвариант считается неподтверждённым, даже когда демонстрация один раз завершилась успешно.

6. Три связанных контура управления

Первый контур — software delivery: commit → CI → image → deployment → telemetry. Второй — model lifecycle: data → features → training run → evaluation → registry → serving → delayed quality. Третий — governance: назначение системы → риски → policy → evidence → approval → review. Ошибка часто возникает на стыке контуров: новый image запущен, но alias указывает на другую модель; качество модели приемлемо, но данные собраны с нарушением временной доступности.

7. Граф доказательств

Итоговый release manifest является индексом графа доказательств. Узлы — identities и отчёты; рёбра — проверяемые связи: этот commit породил этот image, этот run использовал эту версию данных, этот deployment обслуживает эту model version, этот dashboard относится к этому окну. Разрыв связи опаснее отсутствия красивого отчёта, потому что делает вывод непроверяемым.

8. Классификация отказов

Для первичной диагностики удобно использовать четыре измерения:

1. слой: data, feature, model, orchestration, API, infrastructure, observability или governance;
2. время: до выпуска, при deployment, сразу после запуска или после накопления drift;
3. влияние: недоступность, неверные ответы, деградация качества, утечка данных или потеря аудита;
4. обнаружение: test, gate, alert, пользователь, аудит или случайная проверка.

Такая запись лучше фразы «сломалась модель»: она задаёт следующий проверочный шаг.

9. Отказ данных

Примеры: изменилась схема, дубли выросли, временная зона стала иной, source задержался, лицензия или право использования не подтверждены. Проверяются data contract, freshness, объём, ключи, временные границы и data card. Автоматическое обучение на неполной выборке запрещается, если policy не описывает допустимую деградацию и способ её обнаружения.

10. Отказ признаков

Наиболее опасен тихий training-serving skew: pipeline обучения и API вычисляют похожий признак по разным правилам. Второй класс — temporal leakage, когда offline join использует сведения из будущего. Доказательства: единая функция или контракт вычисления, point-in-time test, timestamp available_at, TTL и сравнение распределений offline/online.

11. Отказ эксперимента и оценки

Высокая метрика не означает корректный эксперимент. Проверяются baseline, split, seed, набор метрик, uncertainty, групповые срезы и воспроизводимость run. Если candidate победил только на одном случайном разбиении или за счёт недоступного признака, проблема возникла до Registry, хотя проявится после выпуска.

12. Отказ Registry и release identity

Alias `champion` является управляемым указателем, а не исторической identity. Диагностика начинается с resolved exact version, run ID, tags и release manifest. Если manifest хранит только alias, нельзя доказать, какая модель обслуживала запрос в момент инцидента. Аналогично image tag `latest` не даёт воспроизводимой связи с commit.

13. Отказ API и контракта

Сервис может быть доступен, но неверно интерпретировать единицы измерения, обязательные поля или категории. Проверяются OpenAPI version, Pydantic validation, отрицательные contract tests, request ID, совместимость model/image и отсутствие raw features в логах. HTTP 200 не является достаточным критерием корректности prediction.

14. Отказ оркестрации

Типичные причины: неидемпотентный task, неправильный retry, частично записанный артефакт, потеря параметров запуска и передача большого объекта через внутреннее состояние оркестратора. Безопасный pipeline пишет внешние versioned artifacts, различает retryable и terminal error и не продвигает модель после неполного run.

15. Отказ инфраструктуры и capacity

Рост latency может быть вызван CPU throttling, памятью, очередью, зависимостью или размером batch, а не моделью как таковой. Нужны percentile latency, error rate, saturation, requests/limits, steady и burst profiles. Масштабирование не исправляет неправильный алгоритм и иногда лишь увеличивает стоимость ошибочных запросов.

16. Отказ наблюдаемости

«Нет alert» не означает «нет проблемы». Dashboard может показывать неправильный период, label может создавать неконтролируемую cardinality, метрика качества может не иметь coverage, а лог — терять request ID. Наблюдаемость также тестируется: известный synthetic request должен появиться в метрике, логе и, при наличии, trace с согласованной identity выпуска.

17. Отказ безопасности и цепочки поставки

Проверяются секреты, permissions workflow, версии зависимостей, источник image и сторонних Actions. Full-length commit SHA делает ссылку на Action неизменяемой, но команда всё равно должна проверить происхождение и назначение кода. Attestation усиливает provenance артефакта, но не доказывает качество модели, отсутствие уязвимостей или корректность бизнес-решения.

18. Отказ governance

Governance failure возникает, когда технически успешный выпуск не имеет владельца риска, срока исключения, основания использования данных или процедуры вывода системы из эксплуатации. Рамка NIST AI RMF полезна как цикл `Govern → Map → Measure → Manage`: контекст риска задаётся до измерения, а результат измерения должен вести к управляемому решению, а не оставаться отдельным отчётом.

19. Человеческие и организационные факторы

Ошибку редко объясняет один человек. Системные факторы включают неясные роли, противоречивые приоритеты, отсутствие времени на review, перегруженный runbook, недоступный staging и стимул скрыть неуспешную попытку. Blameless подход не отменяет ответственности: он переносит внимание с поиска виновного на изменение условий, при которых ошибка стала возможной и незаметной.

20. Каскадный отказ: итоговый кейс

Рассмотрим цепочку: источник опоздал; freshness gate был вручную принят без срока; training job создал модель на неполных данных; alias перемещён; image с новым API пересобран отдельно; alert по coverage отсутствует; rollback вернул только alias и оставил несовместимый API. Каждый компонент «работал», но система нарушила несколько связей и усилила исходную проблему.

21. От симптома к гипотезе

Нельзя сразу менять модель по первому сигналу. Зафиксируйте время и влияние, затем identities deployment, image, model и data window. Сформулируйте проверяемые гипотезы: «latency выросла после image X», «доля неизвестных категорий выросла в source Y», «alias сменился без compatible API». Для каждой гипотезы укажите evidence, способ опровержения и безопасный следующий шаг.

22. Triage и приоритет

Первая цель triage — ограничить влияние, а не найти идеальную root cause. Приоритет определяется масштабом воздействия, обратимостью, безопасностью данных и скоростью ухудшения. Команда назначает incident lead, технических исполнителей и канал фиксации решений. Одновременные несогласованные изменения усложняют диагностику и должны быть остановлены.

23. Containment

Возможные действия: остановить продвижение, отключить проблемный source, переключить на безопасный baseline, ограничить трафик, вернуть предыдущую совместимую пару model/image, перевести результат в режим рекомендации или временно прекратить prediction. Containment выбирается так, чтобы не стереть evidence и не создать более опасное состояние.

24. Rollback или roll forward

Rollback предпочтителен, когда предыдущая версия известна, совместима и её возврат быстрее анализа. Roll forward разумен для локальной безопасной ошибки с коротким исправлением и высоким риском обратной миграции. Решение содержит trigger, from/to identities, approver, ожидаемый результат и post-change checks. Оба варианта являются новым управляемым изменением, а не отменой истории.

25. Проверка восстановления

Завершение команды deployment не закрывает incident. Проверяются readiness, smoke prediction, resolved model version, error rate, latency, ключевой бизнес-инвариант и отсутствие дальнейшего повреждения данных. Время восстановления фиксируется по достижению проверенного состояния, а не по моменту нажатия кнопки rollback.

26. Postmortem как инженерный артефакт

Postmortem содержит impact, timeline, detection, response, contributing factors, что сработало, что не сработало и action items. Он не переписывает историю и не маскирует неопределённость. Практика Google SRE рекомендует blameless анализ: корректируются технология, процесс и обучение, а не формулируется действие «быть внимательнее».

27. Непосредственная причина и системные факторы

Фраза «неверный config» описывает событие, но не объясняет, почему config прошёл review, gate и deployment. Полезные вопросы: какое допущение оказалось неверным; какой контроль отсутствовал; почему сигнал не был замечен; что усилило влияние; что помогло восстановлению. Root cause может быть несколько, а линейная история часто скрывает взаимодействие факторов.

28. Action items

Хороший action item имеет владельца, срок, приоритет и проверяемый критерий закрытия. Он либо предотвращает класс отказов, либо ускоряет обнаружение, либо уменьшает impact, либо улучшает восстановление. «Добавить тест» уточняется входом, ожидаемым отказом и местом gate. Просроченный пункт возвращается в risk register, а не исчезает из документа.

29. Технический долг и risk register

Технический долг — не список всего несовершенного. Запись содержит решение или ограничение, последствие, вероятность, blast radius, сигнал наступления, mitigation, owner и review date. Приоритет получают элементы, которые нарушают инвариант, затрудняют восстановление или повторяются в инцидентах. Косметический рефакторинг не должен вытеснять проверяемый safety gap.

30. Метрики развития системы

Одна accuracy и число deployment не описывают здоровье MLOps. Набор включает delivery throughput и stability, SLO сервиса, freshness/coverage данных, delayed model quality, долю успешных gates, время обнаружения и проверенного восстановления, повторяемость инцидентов и закрытие action items. Метрики используются совместно: ускорение выпуска при росте неуспешных изменений не является зрелостью.

31. Уровни зрелости учебного проекта

Уровень	Наблюдаемое состояние
0. Ручной	Результат существует только на машине автора, identities отсутствуют
1. Воспроизводимый	Среда, данные и обучение можно повторить по инструкции
2. Проверяемый	Tests, contracts, CI и evidence подтверждают качество конкретного commit
3. Управляемый	Registry, release gates, observability, approvals и rollback связаны
4. Обучающийся	Incidents, drift и postmortems меняют policy, backlog и архитектуру

Уровень не присваивается по наличию инструмента. Он подтверждается воспроизводимой демонстрацией и тем, что студент способен объяснить ограничения контроля.

32. Архитектурный review

Review начинается не со схемы компонентов, а с цели, пользователей и unacceptable outcomes. Затем команда показывает границы доверия, data flow, identities, gates, observability и recovery. Для каждого критического решения отвечают:

1. какое требование оно обеспечивает;
2. какая альтернатива рассматривалась;
3. какое evidence подтверждает выбор;
4. какой failure mode остаётся;
5. как обнаружить и безопасно восстановить систему.

33. Выбор следующего улучшения

После защиты не нужно одновременно внедрять Kubernetes, feature store и новый registry. Выберите один bottleneck с доказанным влиянием. Если большая часть времени уходит на восстановление среды, улучшайте reproducibility; если инциденты поздно обнаруживаются — telemetry; если release нельзя связать с моделью — identities и manifest; если риск неизвестен — сначала measurement, а не новый инструмент.

34. Подготовка к экзаменационному контролю

Ответ строится как инженерное рассуждение, а не перечисление определений:

1. определить границу и цель системы;
2. назвать входы, выходы и identities;
3. сформулировать инвариант и риск;
4. указать preventive и detective controls;
5. назвать evidence и owner решения;
6. описать отказ, containment и recovery;
7. объяснить ограничение и следующий шаг.

Если вопрос относится к конкретному инструменту, сначала объясняется функция в системе, затем реализация этим инструментом.

35. Пример экзаменационного кейса

После выпуска error rate не изменился, но через две недели снизилась delayed precision для одного направления аэропорта. Coverage меток — 55 %, PSI вырос, latency стабильна. Студент должен отделить факт от гипотезы, проверить сегмент и coverage, сопоставить data window и release identity, выбрать между наблюдением, retraining и rollback, задать gate и назвать остаточный риск. Автоматический retraining только по PSI не является достаточным ответом.

36. Индивидуальная итоговая защита

Каждый участник связывает личный артефакт с архитектурным инвариантом и release tag команды. За 3–4 минуты студент показывает evidence, выполняет диагностическую операцию и отвечает на вопрос по соседнему компоненту. Общая презентация не заменяет личную часть; количество commit не заменяет содержательный diff, review, тест, runbook или решение о риске.

37. Ретроспектива команды

Команда отвечает на четыре вопроса:

- какой контроль дал наибольшую уверенность и чем это доказано;
- где было больше всего ручной работы или неопределённости;
- какой отказ проект обнаруживает поздно или не обнаруживает;
- какое одно улучшение команда реализовала бы следующим и по какому критерию проверила эффект.

Ретроспектива не влияет на уже зафиксированную историю выпуска, но создаёт backlog для следующей версии и показывает зрелость инженерного мышления.

38. Сценарий занятия для преподавателя

1. 0–10 минут: восстановить три контура управления и архитектурные инварианты.
2. 10–25 минут: разобрать failure taxonomy на примерах студентов.

3. 25–45 минут: провести каскадный incident от delayed source до несовместимого rollback.
4. 45–60 минут: группы формируют triage, containment и recovery с evidence.
5. 60–75 минут: превратить timeline в blameless postmortem и action items.
6. 75–85 минут: разобрать структуру экзаменационного ответа и индивидуальной защиты.
7. 85–90 минут: exit ticket и фиксация следующего улучшения проекта.

39. Демонстрация преподавателя

Возьмите manifest ЛР16 и подготовьте копию incident-пакета. Покажите:

1. исходный alert и границу известного;
2. поиск source commit, image и exact model version;
3. гипотезу, которую telemetry опровергает;
4. решение rollback/roll forward с критерием;
5. post-change smoke и проверку model identity;
6. короткий postmortem с двумя action items;
7. перенос незакрытого риска в backlog с owner и review date.

Не меняйте production-like учебные артефакты во время демонстрации: используйте копию spec/evidence и сохраните исходный release неизменным.

40. Задание аудитории

Команды получают разные симптомы: рост p95, падение coverage, несовпадение alias, schema error или утечку признака. За семь минут они заполняют таблицу:

Поле	Ответ команды
Наблюдаемый симптом	Что известно без предположений
Impact	Пользователи, решения, данные, период
Identities	Commit, data, run, model, image, environment
Две гипотезы	Какую evidence запросить и чем опровергнуть
Containment	Как ограничить влияние без потери evidence
Recovery	Rollback или roll forward и post-check
Action item	Owner, срок и критерий закрытия

Затем другой участник команды выполняет review: ищет необоснованный вывод, непроверяемую identity или опасное действие.

41. Вопросы аудитории

1. Почему зелёный CI не доказывает корректность production decision?
2. Чем симптом отличается от contributing factor?
3. Почему rollback может усилить incident?
4. Как проверить, что dashboard относится к нужному release и окну?
5. Когда отсутствие alert является отказом наблюдаемости?
6. Почему blameless postmortem не отменяет ответственность?
7. Как отличить полезный action item от формальной записи?

8. Какие риски нельзя закрыть автоматическим gate?
9. Как выбрать между новым инструментом и улучшением текущего контроля?
10. Как доказать достижение четвёртого уровня зрелости?

42. Exit ticket

Каждый студент индивидуально сдаёт три коротких утверждения:

1. самый опасный неподтверждённый инвариант проекта;
2. evidence, которое могло бы его подтвердить или опровергнуть;
3. одно улучшение с owner, сроком и измеримым критерием.

Совпадающие формулировки без связи с личным артефактом не принимаются. Exit ticket используется как последний вход преподавателя при выборе индивидуального вопроса на защите.

43. Итог курса

MLOps — это дисциплина управления изменениями ML-системы. Зрелая команда не обещает отсутствие ошибок; она делает состояние воспроизводимым, риски видимыми, решения проверяемыми, выпуск обратимым, а инциденты — источником улучшения. Итоговый критерий курса: студент способен не только запустить pipeline, но и обосновать, почему системе можно доверять в известных границах.

44. Официальные источники

- Google SRE, Postmortem Culture: https://sre.google/resources/book-update/postmortem-culture/
- Google SRE, Incident Management Guide: https://sre.google/resources/practices-and-processes/incident-management-guide/
- NIST AI RMF Core: https://airc.nist.gov/airmf-resources/airmf/5-sec-core/
- NIST AI RMF Playbook: https://airc.nist.gov/airmf-resources/playbook/
- GitHub Actions Secure Use Reference: https://docs.github.com/en/actions/reference/security/secure-use
- GitHub Artifact Attestations: https://docs.github.com/en/actions/concepts/security/artifact-attestations